Document number: P1101R0
Date: 2018-05-22
Project: SG1, EWG
Reply-to: Mikhail Maltsev <mikhail.maltsev@arm.com>
Richard Sandiford <richard.sandiford@arm.com>

Vector Length Agnostic SIMD

1. Introduction

This paper proposes extensions to the C++ Standard and the Parallelism v2 TS [1] enabling the support for vector length agnostic SIMD extensions such as the Arm Scalable Vector Extension (SVE) and the RISC-V Vector Extension. This paper is mostly based on the SVE.

The Scalable Vector Extension (SVE) [2] is an extension of the ARMv8-A A64 instruction set developed to target HPC workloads. Unlike traditional SIMD architectures, which define a fixed size for their vector registers, SVE only specifies a maximum size. This freedom of choice is done to enable different Arm-based CPU vendors develop their own implementation, targeting specific workloads and technologies which could benefit from a particular vector length.

A key goal of SVE is to allow the same program image to be run on any implementation of the architecture (which might implement different vector lengths), so it includes instructions which permit vector code to adapt automatically to the current vector length at runtime.

2. Sizeless types

This proposal introduces a notion of *sizeless* types into the C++ type system. The size of an object of a sizeless type is not known at compile time, but becomes known at run time and does not change throughout the object lifetime.

Objects of sizeless types can be allocated on the stack and in certain CPU registers, thus such objects are only allowed to have automatic storage duration. Types of function parameters and return types can be sizeless.

It is allowed to create pointers and references to sizeless types.

Sizeless types retain some of the restrictions of the standard-defined incomplete types:

- The argument to `sizeof` and `alignof` cannot be a sizeless type, or an object of sizeless type.
- It is not possible to perform arithmetic on pointers to sizeless types.
- Sizeless type cannot be used as an operand of a new-expression.
- Members of unions, structures and classes cannot have sizeless type.
- It is not possible to throw or catch objects of sizeless type.
- Standard library containers like `std::vector` cannot have a sizeless `value_type`.

3. Core language design considerations

We realize that sizeless types are a feature implemented only in a limited set of architectures, and this feature is only used in SIMD computations. Therefore we would prefer to minimize the impact on the C++ standard for its users and implementors.

3.1. Interaction with `sizeof`

Since it is not possible to evaluate the size of an object of a sizeless type at compile time, we see two possible ways of sizeof behavior for this case

1. Evaluate the size at run time
2. Make `sizeof` ill-formed for sizeless types

In this paper we propose the latter, since sizeof is currently a core constant expression and changing this would make this proposal very invasive.

3.2. Built-in sizeless types

The set of built-in sizeless types is implementation-defined (and implementations that do not support any sizeless types will still be considered conforming). We do not propose to standardize the names of built-in sizeless types. The users are encouraged to use the SIMD library types instead.

3.3. Sizeless data members

There are multiple possible options for sizeless data members, such as:

1. Do not allow sizeless objects to be class members
2. Allow sizeless object to be the last member of a class
3. Allow embedding sizeless objects at arbitrary locations
4. Make sizeless classes follow an entirely new set of rules (for example, allow the implementation to reorder data members of sizeless classes)

Some support of sizeless members is required to implement this proposal at least as an implementation detail since `simd` and `simd_mask` are standard library templates rather than built-in types, and thus need to have an implementation.

This proposal takes the most conservative approach 1, which can be extended later. Specifically, we do not specify any mechanism that would allow users to define classes with sizeless data members. SIMD library implementors will need to use compiler-specific features not explicitly defined in the standard in order to implement templates such as `simd`, `simd_mask` and `where_expression` for sizeless SIMD types.

This resembles the C++ atomics: while the `std::atomic` template is merely a library type, it cannot be implemented without proper support in the core language and compiler-specific built-ins.

The Arm HPC Compiler team is experimenting with option 4 mentioned above. Specifically, the HPC Compiler supports a new *class-key*, `__sizeless_struct`. Classes defined as `__sizeless_struct` can include sizeless data members, and their size and layout (offsets of data members) only becomes known at run time. The use of such classes is more restricted compared to normal classes (for example querying member offsets using the `offsetof` macro is not allowed).

4. Changes to Parallelism TS

The general direction proposed in this paper is to allow using sizeless SIMD types in the same way the sized types are used, and at the same time not to restrict the users who do not care about the architectures with vector length-agnostic SIMD.

Specifically, we propose to add a new kind of ABI tags and avoid changing the requirements for the instantiations of SIMD library templates that do not use the sizeless ABI tags.

An alternative (described in [4]) would be to define a separate set of SIMD library types for the vector length agnostic SIMD, such as `sizeless_simd`, `sizeless_simd_mask`, etc. The TS should also define a common set of operations supported by both the sizeless types, and the sized ones (for example, in form of concepts).

4.1. ABI tags

We need to provide a new kind of ABI tags for sizeless types. Since currently the TS allows implementations define extended ABI tags, we cannot assume that there will be only a single tag for sizeless SIMD type.

It is implementation-defined whether sizeless SIMD types are supported.

The user needs a way to distinguish the ABI tags of sizeless types from the sized ones. The most simple way to do this is to add a static boolean member to the ABI tag classes, i.e.:

```
struct scalar {
  static constexpr bool is_sized = true;
};
```

4.2. Changes to the `size` member functions

Since sizeless types can only be used in restricted contexts (for example, under the current proposal, sizeless objects are not allowed to have static storage duration), we need to explicitly define the set of SIMD library types that are allowed to be defined as sizeless and the conditions under which they are defined as sizeless. Specifically, the types

- `simd<T, Abi>`
- `simd_mask<T, Abi>`
- `const_where_expression<simd_mask<T, Abi>, simd<T, Abi>>`
- `where_expression<simd_mask<T, Abi>, simd<T, Abi>>`

are sizeless iff `Abi::is_sized` is `false`.

4.3. SIMD type constructors

Class template `simd` supports 4 kinds of constructors: conversion constructor, broadcast constructor, generator constructor and load constructor. Only the conversion and generator constructors are affected by sizeless types.

The conversion constructor only allows conversions from and to `fixed_size<N>` ABI tags. We propose to add a new overload for sizeless types.

The generator constructor is defined in such a way that it is only implementable for constexpr size (the type of the argument passed to `gen` is `std::integral_constant`). We propose to change it to `size_t`.

4.4. Non-member functions

`split` and `concat` inherently rely on width being a compile-time constant. We propose to disable these functions for sizeless types. Alternatively, we could allow using them with sizeless types by providing additional overloads that do not perform return type deduction, and require the user to provide objects of correct widths (otherwise the behavior is undefined).

`simd_cast`, `static_simd_cast` and `to_fixed_size` require changes in wording to support sizeless types. Casting between fixed-sized and sizeless SIMD types can be made possible by saying that the behavior is undefined if the actual size of `From` is less than the size of `To` at run time. For now we propose to disable casting between sized and sizeless types.

5. Implementation experience

The specification defining the SVE extension [2] has been published. Although it is currently in “beta” stage, it is highly unlikely that it will be changed in a way that would affect this proposal.

The “ARM C Language Extensions for SVE” specification [3] defining the extensions for C and C++ standards required for SVE intrinsics support is also available.

Initial SVE support has been upstreamed both in GCC and LLVM. These toolchains support use of SVE instructions in assembly language. GCC can generate SVE instructions in auto-vectorized code. Support for SVE types and intrinsics in C and C++ is currently available in an LLVM-based toolchain provided by Arm.

Upstreaming the intrinsics support in GCC and LLVM is work-in-progress.

Hardware implementations of SVE-enabled cores are currently not available commercially but are expected.

6. Questions proposed for discussions and polls

Core language:

1. Should we proceed with this proposal and sizeless types?
2. `sizeof(sizeless_t)`: ill-formed vs. non-constant?
3. Sizeless class members (in user code)?

Parallelism TS:

1. Sizeless specializations vs separate set of primary templates?
2. `simd` generator constructor: disable for sizeless types or change the parameter type of `gen`?
3. Allow conversion between sizeless and fixed-size SIMD types?

4. Allow concatenation and splitting of sizeless SIMD types (by requiring the user to provide objects of correct widths)?

7. C++ Standard Wording

Change in [basic.def]/5:

> A program is ill-formed if the definition of any object gives the object an ~~incomplete~~**indefinite** type.

Add new paragraph after [basic.def]/5:

> **A program is ill-formed if any declaration of an object gives it both a sizeless type and either static or thread-local storage duration.**

Change in [basic.types]/5:

> A class that has been declared but not defined, an enumeration type in certain contexts ([dcl.enum]), or an array of unknown bound or of ~~incomplete~~**indefinite** element type, is an ~~*incompletely-defined*~~***indefinite*** *object type*. ~~Incompletely-defined~~**Indefinite** object types and cv void are ~~*incomplete*~~***indefinite*** *types* ([basic.fundamental]). Objects shall not be defined to have an ~~incomplete~~**indefinite** type.
>
> **Object and void types are further partitioned into *sized* and *sizeless*; all basic and derived types defined in this standard are sized, but an implementation may provide additional sizeless types.**
>
> **An object or void type is said to be *complete* if it is both sized and definite; all other object and void types are said to be *incomplete*. The term *completely-defined object type* is synonymous with *complete object type*.**
>
> **Arrays and enumeration types are always sized, so for them the term *incomplete* is equivalent to (and used interchangeably with) the term *indefinite*.**

Change in [basic.types]/7:

> [Note: The rules for declarations and expressions describe in which contexts ~~incomplete~~**indefinite** types are prohibited. — end note]

Change in [basic.fundamental]/9:

> A type cv `void` is ~~an incomplete~~**sized indefinite** type that cannot be completed **(made definite)**; …

Change in [basic.compound]/3:

> … Pointers to incomplete types **(including indefinite types)** are allowed although there are restrictions on what can be done with them. …

Change in [basic.lval]/9:

> Unless otherwise indicated ([expr.call]), a prvalue shall always have ~~complete~~**definite** type or the void type. A glvalue shall not have type cv void. [Note: A glvalue may have ~~complete~~**definite** or ~~incomplete~~**indefinite** non-void type. Class and array prvalues can have cv-qualified types; other prvalues always have cv-unqualified types. See [expr.prop]. — end note]

Change in [conv.lval]/1:

> A glvalue of a non-function, non-array type `T` can be converted to a prvalue. If `T` is an ~~incomplete~~**indefinite** type, a program that necessitates this conversion is ill-formed. If `T` is a non-class type, the type of the prvalue is the cv-unqualified version of `T`. Otherwise, the type of the prvalue is `T`.

Change in [expr.call]/7:

> … When a function is called, the parameters that have object type shall have ~~completely-defined~~**definite** object type. [Note: this still allows a parameter to be a pointer or reference to an ~~incomplete~~**indefinite** class type. However, it prevents a passed-by-value parameter to have an ~~incomplete~~**indefinite** class type. — end note] …

Change in [expr.unary.op]/1:

> … [Note: Indirection through a pointer to an ~~incomplete~~**indefinite** type (other than cv void) is valid. The lvalue thus obtained can be used in limited ways (to initialize a reference, for example); this lvalue must not be converted to a prvalue, see [conv.lval]. — end note]

Change in [expr.delete]/2:

> If the operand has a class type, the operand is converted to a pointer type by calling the above-mentioned conversion function, and the converted operand is used in place of the original operand for the remainder of this subclause. **The type of the operand must now be a pointer to a sized type, otherwise the program is ill-formed.** …

Change in [dcl.array]/1:

> … `T` is called the array element type; this type shall not be a reference type, cv void, **a sizeless type,** a function type or an abstract class type. …

Change in [class.static.data]/2:

> The declaration of a non-inline static data member in its class definition is not a definition and may be of an ~~incomplete~~**sized indefinite** type other than cv void.

Add new paragraph after [class.static.data]/7:

> **A static data member shall not have sizeless type.**

Change in [temp.arg.type]/2:

> … [Note: A template type argument may be an ~~incomplete~~**indefinite** type. — end note]

Change in [meta.unary.prop]/3:

> For all of the class templates X declared in this subclause, instantiating that template with a template-argument that is a class template specialization may result in the implicit instantiation of the template argument if and only if the semantics of X require that the argument is a ~~complete~~**definite** type.

Replace all occurrences of “complete” with “definite” in the table in [meta.unary.prop]/4.

Replace all occurrences of “complete” with “definite” in the table in [meta.rel]/2.

8. Parallelism TS v2 Wording

Change in [parallel.simd.general]/1:

> The data-parallel library consists of data-parallel types and operations on these types. A data-parallel type consists of elements of an underlying arithmetic type, called the element type. ~~The number of elements is a constant for each data-parallel type and called the width of that type.~~

Add paragraph after [parallel.simd.general]/2:

The number of elements of a data-parallel object does not change during object lifetime and is called the width of the corresponding data-parallel type.

Change in [parallel.simd.syn]:

```
struct scalar ~~{}~~;
template<int N> struct fixed_size ~~{}~~;
```

Change in [parallel.simd.abi]:

```
struct scalar ~~{};~~
  static constexpr bool is_sized = true;
};

template<int N> struct fixed_size ~~{};~~
  static constexpr bool is_sized = true;
};
```

Add paragraph after [parallel.simd.abi]/8:

> **An implementation shall define the static constexpr boolean data member `is_sized` in each extended ABI tag. The width of the `simd<T, Abi>` specializations for which `Abi::is_sized` is `false` is not known at compile time.**

Change in [parallel.simd.overview]:

```
static ~~constexpr~~see below size_t size() noexcept;
```

Change in [parallel.simd.overview]/1-2:

> The class template simd is a data-parallel type. The width of a given `simd` specialization is ~~a constant expression,~~ determined by the template parameters **and the platform**.
>
> Every specialization of `simd` shall be a ~~complete~~**definite** type. The specialization `simd<T, Abi>` is supported if `T` is a vectorizable type and
>
> - Abi is `simd_abi::scalar`, or
> - Abi is `simd_abi::fixed_size<N>`, with `N` constrained as defined in [simd.abi].
>
> If Abi is an extended ABI tag, it is implementation-defined whether simd<T, Abi> is supported. [Note: The intent is for implementations to decide on the basis of the currently targeted system. — end note]
>
> **if `Abi::is_sized` is `false` and the specialization `simd<T, Abi>` is supported, the specialization shall be a sizeless type, otherwise it shall be a sized (complete) type.**

Change in [parallel.simd.overview]/4:

```
static constexpr size_t size() noexcept;
static size_t size() noexcept;
```

> *Returns:* The width of `simd<T, Abi>`.

> ***Remarks:*** **This function is declared `constexpr` iff `abi_type::is_sized` is `true`**

Add paragraphs after [parallel.simd.ctor]/4:

> **`template <class U> simd(const simd<U, abi_type>& x);`**
>
> ***Effects:*** **Constructs an object where the i-th element equals `static_cast<T>(x[i])` for all `i` ∈ `[0, size())`.**
>
> ***Remarks:*** **This constructor shall not participate in overload resolution unless**
>
> - **`abi_type::is_sized` is `false`, and**
> - **every possible value of U can be represented with type value_type, and**
> - **if both `U` and `value_type` are integral, the integer conversion rank [conv.rank] of `value_type` is greater than the integer conversion rank of `U`.**

Change in [parallel.simd.ctor]/8-11:

> `template <class G> simd(G&& gen);`
>
> Effects: Constructs an object where the i-th element is initialized to `gen(~~integral_constant<size_t, i>()~~i)`.
>
> Remarks: This constructor shall not participate in overload resolution unless `simd(gen(~~integral_constant<size_t, i>()~~size_t{}))` is well-formed ~~for all i ∈ [0, size())~~. The calls to `gen` are unsequenced with respect to each other. Vectorization-unsafe standard library functions may not be invoked by gen ([algorithms.parallel.exec]).

Change in [parallel.simd.casts]/1-6:

> `template<class T, class U, class Abi>` *`see below`* `simd_cast(const simd<U, Abi>& x)`
>
> Let `To` identify `T::value_type` if `is_simd_v<T>` is `true`, or `T` otherwise.
>
> *Returns:* A simd object with the i-th element initialized to `static_cast<To>(x[i])` for all `i ∈ [0, size())`.
>
> *Throws:* Nothing.
>
> *Remarks:* The function shall not participate in overload resolution unless every possible value of type `U` can be represented with type `To`, and either
>
> - `is_simd_v<T>` is `false`, or
> - **`T::abi_type` is `Abi`, and `U` is `T` or**
> - **`Abi::is_sized` is `true`, and `T::abi_type::is_sized` is `true`, and** `T::size() == simd<U, Abi>::size()` is `true`.
>
> The return type is
>
> - `T` if `is_simd_v<T>` is `true`, otherwise
> - `simd<T, Abi>` if `U` is `T`, otherwise
> - `simd<T, simd_abi::fixed_size<simd<U,Abi>::size()>>`

Change in [parallel.simd.casts]/7-12:

> `template<class T, class U, class Abi>` *`see below`* `static_simd_cast(const simd<U, Abi>& x)`

> Let `To` identify `T::value_type` if `is_simd_v<T>` is `true`, or `T` otherwise.
>
> *Returns:* A simd object with the i-th element initialized to `static_cast<To>(x[i])` for all `i ∈ [0, size())`.
>
> *Throws:* Nothing.
>
> *Remarks:* The function shall not participate in overload resolution unless either
>
> - `is_simd_v<T>` is `false`, or
> - **`T::abi_type` is `Abi`, and `U` is `T` or `U` and `T` are integral types that only differ in signedness, or**
> - **`Abi::is_sized` is `true`, and `T::abi_type::is_sized` is `true`, and** `T::size() == simd<U, Abi>::size()` is `true`.
>
> The return type is
>
> - `T` if `is_simd_v<T>` is `true`, otherwise
> - `simd<T, Abi>` if `U` is `T` or `U` and `T` are integral types that only differ in signedness, otherwise
> - `simd<T, simd_abi::fixed_size<simd<U, Abi>::size()>>`

Add paragraph after [parallel.simd.casts]/14:

```
template<class T, class Abi>
fixed_size_simd<T, simd_size_v<T, Abi>> to_fixed_size(const simd<T, Abi>& x)
noexcept;
template<class T, class Abi>
fixed_size_simd_mask<T, simd_size_v<T, Abi>> to_fixed_size(const simd_mask<T, Abi>&
x) noexcept;
```

> *Returns:* A data-parallel object with the i-th element initialized to `x[i]` for all `i ∈ [0, size())`.
>
> ***Remarks:* These functions shall not participate in overload resolution unless `Abi::is_sized` is `true`**

Change in [parallel.simd.mask.overview]:

```
static ~~constexpr~~see below size_t size() noexcept;
```

Change in [parallel.simd.mask.overview]/1-2:

> The class template `simd_mask` is a data-parallel type with the element type `bool`. The width of a given `simd_mask` specialization is ~~a constant expression,~~ determined by the template parameters **and the platform**. Specifically, `simd_mask<T, Abi>::size() == simd<T, Abi>::size()`.
>
> Every specialization of `simd_mask` shall be a ~~complete~~**definite** type. The specialization `simd_mask<T, Abi>` is supported if `T` is a vectorizable type and
>
> - Abi is `simd_abi::scalar`, or
> - Abi is `simd_abi::fixed_size<N>`, with `N` constrained as defined in [simd.abi].
>
> If Abi is an extended ABI tag, it is implementation-defined whether simd_mask<T, Abi> is supported. [Note: The intent is for implementations to decide on the basis of the currently targeted system. — end note] **If `Abi::is_sized` is `false` and the specialization `simd<T, Abi>` is supported, the specialization shall be a sizeless type, otherwise it shall be a sized (complete) type.** If

> `simd_mask<T, Abi>` is not supported, the specialization shall have a deleted default constructor, deleted destructor, deleted copy constructor, and deleted copy assignment.

Change in [parallel.simd.mask.overview]/4:

> ~~`static constexpr size_t size() noexcept;`~~
> **`static size_t size() noexcept;`**
>
> *Returns:* The width of `simd<T, Abi>`.
>
> ***Remarks:*** **This function is declared `constexpr` iff `abi_type::is_sized` is `true`**

9. References

1. N4742 Working Draft, Technical Specification for C++ Extensions for Parallelism Version 2
2. ARM Architecture Reference Manual Supplement — The Scalable Vector Extension (SVE), for ARMv8-A
3. ARM C Language Extensions for SVE
4. P0349R0 Assumptions about the size of datapar